

What LLM Rewriting Costs an Adversarial Attack

Yihang Jiao

University of Illinois Urbana-Champaign

yihangj3@illinois.edu

Abstract

Rewriting an adversarial example with a large language model is an appealing way to make attacks on text classifiers read naturally. This paper measures what that rewriting costs. Starting from word-substitution attacks that already succeed, I find that LLaMA-2 rewrites keep the attack working only 44.5% of the time across 32 model–attack–dataset configurations: more than half of the attacks are lost. The loss is not uniform, and it is governed by the prompt. In a controlled ablation that runs a task-specific and a generic prompt on the same data, the task-specific prompt roughly doubles retention (119/180 vs. 58/180 rewrites, $p = 1.7 \times 10^{-10}$). For sentiment tasks this gain is free, with semantic similarity unchanged (0.713 vs. 0.705); for topic classification it is not, because the prompt that raises the flip rate does so by instructing the model to change the topic, and similarity collapses from 0.691 to 0.361. On sentence-pair tasks a further failure appears: most rewrites collapse the premise/hypothesis structure and are not valid classifier inputs at all, though an explicit structure constraint nearly doubles validity (57.5% vs. 30.0%, $p = 2.8 \times 10^{-5}$). Finally, I measure the generate-and-filter pipeline that this literature proposes and find it retains 15.3% of generations, with the semantic-similarity constraint, not fluency, doing most of the filtering. I also document an evaluation error in an earlier version of this study, in which reported success rates were computed from the attack that preceded rewriting rather than from the rewrites themselves.

1 Introduction

Adversarial attacks on text classifiers expose inputs on which a model’s decision is unstable. Word-substitution attacks such as TextFooler (Jin et al., 2020) and PWWS (Ren et al., 2019) search for synonym replacements that flip a prediction while nominally preserving meaning. Their outputs are effective but often stilted, and the stiltedness matters: a perturbation that no fluent speaker would produce says less about deployment risk than one that reads normally.

This motivates an obvious idea, pursued in a growing body of work: hand the adversarial example to a large language model and ask it to rewrite the text so that it stays adversarial but reads naturally. The appeal is that an LLM can restructure a whole sentence rather than swap tokens, and can therefore produce attacks that are fluent by construction.

I take the idea seriously enough to measure it. My question is not whether LLM rewriting produces fluent text — it plainly does — but what the rewriting costs. Because rewriting is applied to attacks that already succeed, the cost is directly measurable: how many of those attacks survive being rewritten?

My contributions are:

- **Rewriting loses most of the attacks it is applied to.** Across 32 configurations, LLaMA-2 rewrites of successful word-substitution attacks still flip the classifier only 44.5% of the time (Section 3).
- **The prompt, not the dataset, governs the loss.** Running both a task-specific and a generic prompt on the same data doubles retention with the dataset held fixed (Section 4). But for topic classification the gain is bought with genuine semantic change, which disqualifies the resulting texts as adversarial examples.

- **Sentence-pair tasks fail structurally.** Most rewrites merge the two segments and are not valid classifier inputs; an explicit structure constraint helps but does not solve it (Section 5).
- **I measure the generate-and-filter pipeline this literature proposes** and report what each constraint removes (Section 6).
- **I document an evaluation error** in an earlier version of this study, in which the classifier was never run on the rewrites (Section 7). The failure mode is general, and I describe the check that catches it.

2 Experimental Setup

Attacks and victims. I use TextFooler (Jin et al., 2020) and PWWS (Ren et al., 2019) as implemented in TextAttack (Morris et al., 2020), against `bert-base-uncased` and `roberta-base` classifiers fine-tuned on each dataset. Attacks were run over 500 test examples per configuration (277–500 after dataset exhaustion), yielding 190–479 successful attacks per configuration to sample from.

Datasets. Single-sentence tasks: IMDB, SST-2, MR (Rotten Tomatoes), AG News, and CoLA. Sentence-pair tasks: RTE, QNLI, and MRPC. Pair inputs are represented as two segments joined by a `<SPLIT>` marker, following TextAttack’s convention.

Rewriting. Rewrites are produced by `Llama-2-7b-chat-hf` (Touvron et al., 2023) in half precision, with nucleus sampling ($p = 0.9$, $T = 0.8$, 160 new tokens), applied through the model’s chat template. I strip the scaffolding that chat models place at both ends of an answer (“Here is the revised text:” and trailing explanations) before scoring; leaving it in place would inflate perplexity and depress similarity for whichever prompt elicits more chattiness, which is the quantity under study.

Measurements. For each rewrite I record: whether the victim classifier’s prediction differs from its prediction on the original text (*flip*); cosine similarity between sentence embeddings of the original and the rewrite, using `all-MiniLM-L6-v2`; and perplexity under GPT-2-XL. For pair tasks I additionally record whether the rewrite preserved the two-segment structure and is therefore a valid classifier input at all.

The comparison of interest. Rewrites are generated only for examples on which the word-substitution attack already succeeded. The baseline is therefore fixed at 100% by construction, and the quantity to report is what fraction of those attacks *survives* rewriting. This is the comparison the earlier version of this study failed to make (Section 7).

3 Rewriting Loses Most Attacks

I first re-evaluate a stored corpus of rewrites spanning 32 configurations (2 victim architectures $\times$ 2 attack recipes $\times$ 8 datasets), running each victim classifier on the rewritten text.

On the 20 single-sentence configurations, rewrites still flip the classifier on average 44.5% of the time. Since every one of these examples was a successful attack before rewriting, **more than half of the attacks are destroyed by the rewrite**. Retention varies enormously by dataset:

Dataset	Victim	Specific	Generic	p
AG News	BERT	20/30	10/30	0.019
AG News	RoBERTa	19/30	8/30	0.009
IMDb	BERT	22/30	7/30	0.0002
IMDb	RoBERTa	28/30	6/30	$< 10^{-4}$
SST-2	BERT	12/30	13/30	1.000
SST-2	RoBERTa	18/30	14/30	0.438
Pooled		119/180	58/180	1.7×10^{-10}

Table 1: Rewrites that still flip the classifier, by prompt variant, with the dataset and victim held fixed. Two-sided Fisher exact test per cell.

Dataset	Mean	Min	Max
AG News	0.907	0.706	1.000
IMDb	0.764	0.667	0.889
MR	0.268	0.077	0.462
SST-2	0.217	0.105	0.300
CoLA	0.069	0.000	0.133

The spread is a factor of thirteen. It also lines up exactly with an implementation detail: AG News and IMDb were the two datasets for which a hand-written prompt template existed, while MR, SST-2 and CoLA fell through to a generic template. Dataset and prompt are confounded in this corpus, so the next section separates them.

4 Prompt Specificity, With the Dataset Held Fixed

To break the confound I run *both* prompt variants on *every* dataset. The generic prompt is the original fallback template (“Improve the text fluency while altering the classification. . .”). The task-specific prompts follow the shape of the original hand-written ones: for IMDb and AG News these are the originals; for SST-2 I write one in the same shape, so that “has a task-specific prompt” becomes a manipulable variable rather than a property of the dataset. I sample 30 successful TextFooler attacks per cell.

Pooled over 360 rewrites, the task-specific prompt roughly doubles the fraction of attacks that survive: 119/180 versus 58/180 ($p = 1.7 \times 10^{-10}$). The effect holds in 5 of 6 cells and is significant in 4; the two SST-2 cells show no reliable difference.

The gain is not always free. Retention is only half the story, because an adversarial example must also preserve meaning. Splitting by task type makes the difference stark:

	Flip rate	Similarity
<i>Sentiment (IMDb, SST-2)</i>		
specific	0.667	0.713
generic	0.333	0.705
<i>Topic (AG News)</i>		
specific	0.650	0.361
generic	0.300	0.691

Dataset	Victim	Prompt	Valid	Flip
QNLI	BERT	generic	0.433	0.200
QNLI	BERT	structured	0.733	0.300
QNLI	RoBERTa	generic	0.600	0.200
QNLI	RoBERTa	structured	0.833	0.333
RTE	BERT	generic	0.067	0.033
RTE	BERT	structured	0.433	0.133
RTE	RoBERTa	generic	0.100	0.067
RTE	RoBERTa	structured	0.300	0.100
Pooled		generic	0.300	0.125
		structured	0.575	0.217

Table 2: Sentence-pair tasks. *Valid*: fraction of rewrites that preserved the two-segment structure and could be fed to the classifier. *Flip*: fraction of all rewrites that both were valid and flipped the prediction. Structure preservation: $p = 2.8 \times 10^{-5}$ (Fisher exact).

On sentiment tasks the specific prompt doubles the flip rate at no cost in semantic similarity: 0.713 versus 0.705. This is a genuine improvement. On AG News the same doubling comes with similarity collapsing from 0.691 to 0.361 — and the reason is visible in the prompt itself, which instructs the model to change the topic “from world to sports”. A text whose topic has actually been changed is not an adversarial example; it is a different input with a different correct label. **The prompt that best preserves the attack does so, on this task, by destroying the property that makes an attack meaningful.**

This is a trap for the whole generate-and-filter approach: the instruction most effective at producing label flips is the instruction to change the label. Reporting flip rate without a paired semantic constraint will systematically reward it.

5 Sentence-Pair Tasks Fail Structurally

For RTE, QNLI and MRPC the classifier expects two segments. Most rewrites do not supply them: the model merges premise and hypothesis into a single passage, often appending a commentary sentence. In the stored 32-configuration corpus, only 100 of 198 pair-task rewrites preserved the structure; for the two RTE configurations under PWWS, *zero* of 31 rewrites did.

I test whether an explicit constraint fixes this, comparing the generic prompt against one that states the input is a pair, that the two sentences are separated by <SPLIT>, and that the answer must return exactly two sentences in the same order.

Stating the constraint nearly doubles structural validity, from 30.0% to 57.5% ($p = 2.8 \times 10^{-5}$), and raises the overall yield of usable attacks from 12.5% to 21.7%. It does not come close to solving the problem: even when told exactly what format to produce, the model violates it more than 40% of the time. Any pipeline that applies LLM rewriting to structured inputs must check structural validity explicitly, and must report it — otherwise the invalid rewrites are silently scored as failures, or worse, scored at all.

6 What the Filter Actually Trades

Proposals in this area pair generation with a filter that keeps only candidates that flip the label, stay semantically close, and read fluently. I apply such a filter to all 360 single-sentence rewrites and report what each stage removes.

The pipeline retains 15.3% of generations. Two observations matter. First, the *similarity* constraint does

Stage	Kept	Frac.	Sim. / PPL
All rewrites	360	1.000	0.648 / 33.8
+ flips the classifier	177	0.492	0.596 / 28.1
+ similarity ≥ 0.75	64	0.178	0.840 / 27.2
+ perplexity ≤ 150	55	0.153	0.834 / 22.9

Table 3: Cumulative effect of the three constraints. Sim. is mean cosine similarity, PPL median GPT-2-XL perplexity, of the surviving set.

most of the filtering: it removes 113 of the 177 candidates that flip, while the fluency constraint removes only 9 more. This inverts the usual framing, in which fluency is presented as the reason to filter. Second, the surviving set is meaningfully better on the axis that was filtered — mean similarity rises from 0.648 to 0.834 — so the filter does what it claims; it is simply far more expensive than reported.

For context, the word-substitution attacks these rewrites replace are much less fluent: over 428 attacked examples, the median GPT-2-XL perplexity of a TextFooler perturbation is 522, against 22.9 for filtered rewrites. Of 403 successful word-substitution attacks, only 15.4% satisfy the same similarity and fluency constraints. **LLM rewriting buys a large improvement in fluency and pays for it in attack retention.** That is the trade, stated plainly, and neither side of it is small.

7 Threats to Validity, Including One I Made

An earlier version of this study reported that filtered LLaMA-2 generations outperformed word-substitution attacks, with improvements up to 19.31% relative. Those numbers do not survive scrutiny, and the reasons are worth recording because the failure mode is easy to reproduce.

The classifier was never run on the rewrites. The pipeline wrote LLaMA output into a `llama_text` column of the results file, and the reported attack success rate was computed from TextAttack’s `result_type` field. That field describes the *word-substitution* attack, which happened before rewriting. The rewrites therefore never entered the reported numbers at all. Section 3 supplies the missing evaluation.

The headline table compared sample sizes. Each “filtered” figure was the attack’s success rate over 20 examples; each “baseline” figure was the same attack over 277–500 examples. All five rows of the original table reproduce exactly from those two runs. The reported improvement was a difference in sample size.

Filtering happened at a different layer than described. The pipeline did constrain and score generations, but not where Algorithm 1 places it. A similarity constraint, `min_cos_sim = 0.6`, was passed to the attack’s word-substitution step, and perplexity and grammar were computed afterwards as aggregate statistics over each result file. What Algorithm 1 describes — a per-candidate test that keeps or drops each rewrite on all three thresholds — is not in that code, and the values 0.75 and 150 appear nowhere in it. Section 6 implements the per-candidate test and reports what each condition removes.

The check that catches this. The general lesson is narrow and mechanical: *if a pipeline produces new text, the reported metric must be a function of that text*. A one-line assertion — that the evaluated column is the generated column — would have caught all three problems. I suggest reporting, alongside any generate-and-fix result, the number of generations actually scored by the victim model, which makes the omission visible.

Remaining limitations. Rewrites come from a single 7B model; larger or instruction-tuned-for-format models may preserve structure better. I sample 30 examples per cell, adequate for the effect sizes reported but not for small ones, which is why the SST-2 cells are inconclusive rather than negative. Similarity is measured with one sentence encoder, and encoder choice will shift the similarity threshold’s severity. Finally, the task-specific prompts for SST-2 were written by me in the shape of the originals; a different phrasing might move the effect.

8 Conclusion

LLM rewriting of adversarial examples is a trade, not an improvement. It produces text that is dramatically more fluent than word-substitution output (median perplexity 22.9 against 522), and it destroys more than half of the attacks it is applied to. How much it destroys is governed by the prompt: a task-specific prompt roughly doubles retention, and on sentiment tasks it does so at no semantic cost. On topic classification the same prompt succeeds by instructing the model to change the topic, which is not an attack but a different input — a trap that flip-rate-only evaluation will reward. On sentence-pair tasks, most rewrites are not valid classifier inputs at all.

None of this makes the approach unusable. It makes it measurable, which the earlier version of this work was not. The measurements that matter are the ones that were missing: run the classifier on the text you generated, check that structured inputs are still structured, and report semantic similarity next to every flip rate.

Reproducibility

Code, the stored rewrites, and every result table in this paper are available at <https://github.com/yhj3/counterfactual-textattack>. The repository documents which components are original and which were reconstructed, and includes the audit described in Section 7.

References

- Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2020. Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In *AAAI*, volume 34, pages 8018–8025.
- John X. Morris, Eli Lifland, Jin Yong Yoo, Jake Grigsby, Di Jin, and Yanjun Qi. 2020. TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In *EMNLP: System Demonstrations*, pages 119–126.
- Shuhuai Ren, Yihe Deng, Kun He, and Wanxiang Che. 2019. Generating natural language adversarial examples through probability weighted word saliency. In *ACL*, pages 1085–1097.
- Hugo Touvron, Louis Martin, Kevin Stone, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. 2018. HotFlip: White-box adversarial examples for text classification. In *ACL (Short Papers)*, pages 31–36.
- Divyansh Kaushik, Eduard Hovy, and Zachary C. Lipton. 2020. Learning the difference that makes a difference with counterfactually-augmented data. In *ICLR*.